Log analysis on PostgreSQL, Pgbouncer, Patroni and consul Runbook

Intro

Nowadays all informatics services/system has a log mechanism due to can register events from services/system and can be useful for *audit* or *troubleshooting*. The Gitlab database architecture is made up of some components such as: PostgreSQL, pgbouncer, patroni and counsul. This runbook describe the most common errors for each component, what that means and how to anlayze logs for these components.

Nowadays all informatics services/system has a log mechanism due to can register events from services/system and can be useful for *audit* or *troubleshooting*. The Gitlab database architecture is made up of some components such as: PostgreSQL, pgbouncer, patroni and counsul. This runbook describe the most common errors for each component, what that means and how to anlayze logs for these components.

PostgreSQL's log

The PostgreSQL's log can be as verbose as we configure it, you can check the log parameters here. The PostgreSQL log's parameters can be grouping in 3 group(**Where To Log,When To Log,What To Log**), some important parameters to analyze log are:

- Where To Log
  - log_destination: Indicate which methods will be used for logs (stderr, csvlog and syslog), Gitlab use *csvlog*
  - log_directory: Shows where the log files will be, Gitlab use */var/log/gitlab/postgresql*
- When To Log
  - log_min_duration_statement: will log all statements whose runtime exceeds this value in milliseconds
- What To Log
  - log_line_prefix :Is the *printf-style* string that appear at the beginning of each line in logs, Gitlab use *%m [%p, %x]: [%l-1] user=%u,db=%d,app=%a,client=%h*. The meaning of each option is:

```
#   %a = application name
#   %u = user name
#   %d = database name
#   %r = remote host and port
#   %h = remote host
#   %p = process ID
#   %t = timestamp without milliseconds
#   %m = timestamp with milliseconds
```

```
#   %n = timestamp with milliseconds (as a Unix epoch)
#   %i = command tag
#   %e = SQL state
#   %c = session ID
#   %l = session line number
#   %s = session start timestamp
#   %v = virtual transaction ID
#   %x = transaction ID (0 if none)
```

The rest of log's parameters you can check the values with the following query:

```
gitlabhq_production=# select name, setting from pg_settings where name like 'log_%';
            name             |                        setting
-----------------------------+-------------------------------------------------------
log_autovacuum_min_duration | 0
log_checkpoints             | on
log_connections             | on
log_destination             | csvlog
log_directory               | /var/log/gitlab/postgresql
log_disconnections          | on
log_duration                | off
log_error_verbosity         | default
log_executor_stats          | off
log_file_mode               | 0640
log_filename                | postgresql.log
log_hostname                | off
log_line_prefix             | %m [%p, %x]: [%l-1] user=%u,db=%d,app=%a,client=%h
log_lock_waits              | on
log_min_duration_statement  | 1000
log_min_error_statement     | error
log_min_messages            | warning
log_parser_stats            | off
log_planner_stats           | off
log_replication_commands    | off
log_rotation_age            | 0
log_rotation_size           | 0
log_statement               | ddl
log_statement_stats         | off
log_temp_files              | 0
log_timezone                | GMT
log_truncate_on_rotation    | off
logging_collector           | on
(28 rows)
```

PostgreSQL logs are stored in `csv format`. A typical error line for postgresql may look like this:

```
2020-07-14 06:29:40.960 GMT,"gitlab","gitlabhq_production",4898,"127.0.0.1:56244",5f0d451a.1
```

The actual query has been trimed for readability.

Most important columns when dealing with PostgreSQL errors are:

- timestamp: To know the exact moment of the error
- severity level: PostgreSQL has different levels of severity, showed here in increasing order:
  - WARNING: An event that, while not preventing the command to complete, may lead to failures if not addressed. Monitoring for warnings is a good practice in early detection of issues on both the server and application side.
  - ERROR: Failure to execute a command.
  - FATAL: The current session is aborted due to an error. The client may retry.
  - PANIC: All sessions are aborted. This situation affects all the clients.
- error code: Provides more insights about the source of the error. A complete list of error codes can be found here. In the example above, the error code is "*57014*", typified as "*query_canceled*"
- message: A human readable, more verbose message describing the error: "*canceling statement due to statement timeout*"
- hints: Some errors will give hints, clues about the possible solution to this problem.

Some common PostgreSQL errors, what it means and how to solve/address it:

`ERROR,57014,"canceling statement due to statement timeout"` By default, the maximum amount of time that any query can be active is `15 seconds`:

```
gitlabhq_production=# show statement_timeout ;
 statement_timeout
-------------------
 15s
(1 row)
```

After that, the query is automacally cancelled. Superusers and other administrative role does not have this limitation. If you need to extend the timeout, you can set `statement_timeout` to a higher threshold, or use a superuser/administrator role.

`FATAL:the database system is starting up` This error can be seen after a server crash. The startup routine needs to be redo all the transactions that were running before the crash,and the system should be ready in 1 minute or less, depending on the load. Another source of this error is when an application tries to connect to a replica that does not have the hot_standby parameter set to `on`.

`ERROR:duplicate key value violates unique constraint` An application is trying to insert a record that contains an existing value for a unique con-

straint (means a unique index or a primary key). This can because of a sequence (implemented for `serial` columns) has been modified (i.e. has been RESET).

Application code could use the ON CONFLICT clause to avoid this error.

`ERROR: value too long for type character varying(64)` An attempt to write more data than allowed for a column. To correct this, you could issue an `ALTER TABLE table_name ALTER COLUMN column_name varchar(128)` to extend that column to accept up to 128 characters.

`FATAL: terminating connection due to idle-in-transaction timeout`. This is due by the idle_in_transatcion_session_timeout setting, to prevent idle transactions for holding connections.

`FATAL: remaining connection slots are reserved for non-replication superuser connections`. This means that this PostgreSQL instance has no more connections available. This is related to the max_connections setting.

`ERROR: deadlock detected`. This happens when 2 separate connections needs a resource (a table, row, etc) that is being locked by the other, mutually locking each other. One of the connections will be terminated. Verify the application flow to see if this situation can be avoided.

`FATAL: too many connections for role "xxx"` Specific roles can have specific connection limit set. Check the `\du` command to check the current limit for that user, and investigate futher.

```
gitlabhq_production=# \du gitlab
              List of roles
 Role name |   Attributes    | Member of
-----------+-----------------+-----------
 gitlab    | 270 connections | {}
```

To manage role specific limits, an `ALTER role xxx CONNECTION LIMIT nn` may be issued.

For deeper, broad analysis of PostgreSQL's logs, pgbadger is a tool that can be used for. Check the pgbadger Runbook to see how to use it.

pgbouncer's log

The pgbouncer's log can be as verbose as we configure it, you can check the log parameters here. some important parameters to analyze log are:

- log_connections: log successfully logins
- log_disconnections: Log disconnections with reasons
- log_stats: log statistics about queries and I/O traffics (useful to analyzing the server activity)

Common entries from pgBouncer logs are: `gitlabhq_production/gitlab@xxxxx:zzz taking connection from reserve_pool`. This means that the pool is under

heavy use. May be related to a burst of long running queries on the postgres side.

`gitlabhq_production/gitlab@xxxxx:zzzz pooler error: server conn crashed?`. This is usually related to connections being canceled (i.e. timeout) on the postgres side.

`stats: 2933 xacts/s, 3187 queries/s, in 635683 B/s, out 1328827 B/s, xact 1387 us, query 912 us, wait 101 us`. PgBouncer logs his activity every minute. Each column represents values for the last minute:

- xacts/s -> transactions per second
- queries/s -> queries per second
- in B/s -> incoming traffic (in bytes)
- out B/s -> outgoing traffic (in bytes)
- xact us -> transaction duration average (in us)
- query us -> query duration average (in us)
- wait us -> waiting average (in us)

For analyzing pgbouncer's logs can be used this script , the location for pgbouncer logs in Gitlab are: */var/log/gitlab/pgbouncer*

Audit

You must run the script

```
sh scripts/parse_bouncer.sh -f name_of_pgbouncer_log_file
```

You will get an output similar to:

```
* AVG Number of queries/sec by pgbouncer
pgbouncer[43984]: 3615.78
pgbouncer[44022]: 3616.67
pgbouncer[44052]: 3613.77
* AVG of queries times(us) by pgbouncer
pgbouncer[43984]: 805.458
pgbouncer[44022]: 804.081
pgbouncer[44052]: 803.901
* AVG of KB in by pgbouncer
pgbouncer[43984]: 1642.27
pgbouncer[44022]: 1642.55
pgbouncer[44052]: 1640.88
* AVG of KB in by pgbouncer
pgbouncer[43984]: 8096.39
pgbouncer[44022]: 8111.58
pgbouncer[44052]: 8101.35
* Conexions by pgbouncer
pgbouncer[43984]: 56791
pgbouncer[44022]: 56663
```

```
pgbouncer[44052]: 56842
* AVG of conexions time(s) by bouncer
pgbouncer[43984]: 1959.95
pgbouncer[44022]: 1958.89
pgbouncer[44052]: 1959.46
* Conexions by IP
10.220.8.58: 1529
10.220.8.19: 4502
10.220.8.2: 4569
10.220.8.18: 4521
10.220.8.17: 1491
10.220.8.3: 4409
10.220.8.4: 4472
10.220.8.16: 1561
10.220.8.15: 4547
10.220.8.5: 4517
10.220.8.14: 4532
10.220.8.6: 4528
10.220.8.7: 4492
10.220.8.13: 4432
10.220.8.21: 4539
10.220.8.8: 4592
10.220.8.12: 4465
10.220.8.20: 4512
10.220.8.11: 4584
10.220.8.9: 4469
10.220.8.10: 4485
10.220.2.18: 2457
10.220.2.2: 2276
10.220.2.17: 2349
10.220.2.26: 2315
10.220.2.4: 2307
10.220.2.25: 2436
10.220.2.34: 2389
10.220.2.24: 2457
10.220.2.33: 2418
10.220.2.5: 2338
10.220.2.23: 1241
10.220.2.32: 2320
10.220.2.14: 2286
10.220.2.6: 2368
10.220.2.22: 1216
10.220.2.31: 2302
10.220.9.196: 1551
10.220.9.195: 1528
10.220.2.8: 2476
```

```
10.220.2.12: 2316
10.220.2.9: 2508
10.220.9.194: 1503
10.220.2.10: 2301
10.250.10.2: 4
10.220.4.19: 1134
10.220.4.18: 1140
10.220.4.2: 1135
10.220.4.3: 1102
10.220.4.17: 1153
10.220.4.16: 1142
10.220.4.4: 1134
10.220.4.15: 1134
10.220.4.33: 1149
10.220.4.5: 1136
10.220.4.23: 1295
10.220.4.32: 1147
10.220.4.14: 1139
10.220.4.6: 1157
10.220.4.22: 1289
10.220.4.7: 1145
10.220.4.31: 1135
10.220.4.13: 1132
10.220.4.12: 1109
10.220.4.30: 1146
10.220.4.21: 1143
10.220.4.8: 1120
10.220.4.20: 1154
10.220.4.11: 1126
10.220.4.9: 1132
10.220.4.10: 1123
127.0.0.1: 5035
* Conexions by users
gitlabhq_production->170296
* Conexions by DB
pgbouncer->184
chatops->6
gitlab->170081
gitlab-monitor->25
* Error by type
bad packet header: 6
client sent partial pkt in startup phase: 27
* Error by pgbouncer/IP
pgbouncer[43984]-> (nodb)/(nouser)@10.250.8.11: 11
pgbouncer[44022]-> (nodb)/(nouser)@10.250.8.11: 11
pgbouncer[44052]-> (nodb)/(nouser)@10.250.8.11: 11
```

Patroni's log

The Patroni's log can help to analyze the health of PostgreSQL cluster for HA implemented by Patroni here. The logs of patroni are located in */var/log/gitlab/patroni*.

- In Normal behavior, you will see in the logs something like:
  - Master

  ```
  INFO: Lock owner: patroni-01-db-gprd.c.gitlab-production.internal; I am patroni-01-db-g
  INFO: no action.  i am the leader with the lock
  ```

  Meaning this is the leader `patroni-01-db-gprd.c.gitlab-production.internal`

  - Standby

  ```
  INFO: Lock owner: patroni-01-db-gprd.c.gitlab-production.internal; I am patroni-05-db-g
  INFO: does not have lock
  ```

  Meaning this is a standby server `I am patroni-05-db-gprd.c.gitlab-production.internal` following the leader `patroni-01-db-gprd.c.gitlab-production.internal`

- Error

  For analyzing patroni's logs with `error` you can use this script:

  ```
  sh scripts/parse_patroni.sh -f patroni.log.1
    * WARNINGS
  WARNING: Postgresql is not running.->1782
    * ERRORS
  ERROR: Error when reading postmaster.opts->2
  ERROR: postmaster is not running->1780
  ```

Then you must find inside the registers some previous and later lines the cause of the error, for example:

```
FATAL:  data directory "/var/opt/gitlab/postgresql/data11" has invalid permissions
DETAIL:  Permissions should be u=rwx (0700) or u=rwx,g=rx (0750).
```

```
FileNotFoundError: [Errno 2] No such file or directory: '/var/opt/gitlab/postgresql/data11/p
```

Consul's log

Consul is the consensus tool that is integrated to patroni in Gitlab and the logs can be found in */var/log/syslog* with tag [**consul**]

- In Normal behavior, you will see in the logs something like:

  ```
  [WARN]  agent: Check is now critical: check=service:patroni-master
  agent: Check is now critical: check=service:patroni-master
  [INFO]  agent: Synced check: check=service:db-replica:2
  agent: Synced check: check=service:db-replica:2
  ```

The output that Consul "synced", meaning that agent loaded the service, and has successfully registered it in the catalog.

- Error

  If some error happed you will find in the logs the tag [**ERR**] and you can used this script to anlayze:

```
 sh scripts/parse_consul.sh -f syslog.1
  * ERRORS
[ERR] yamux: keepalive failed: session shutdown->2
```